

Splunk SIEM Log Monitoring & Detection Workflow

In this lab, I built a Splunk-based SIEM detection pipeline that mirrors the workflow implemented in Lab 5. The purpose of this lab is to demonstrate that the same attack scenarios and detection logic can be reproduced using a different SIEM platform.

While Lab 5 focused on Wazuh, this lab replaced the SIEM with Splunk Enterprise. Network-based alerts generated by Suricata and host-based SSH authentication logs are ingested, analyzed, and correlated to reconstruct attacker behavior.

Nmap and Hydra attacks are launched from a Kali Linux attacker to simulate reconnaissance and SSH brute-force activity.

The lab environment consists of two virtual machines connected through a VirtualBox host-only network. Kali Linux is used as the attacker system, while Ubuntu Server acts as the target machine and hosts both Suricata and Splunk Enterprise.

Suricata inspects network traffic and writes events to a JSON log file, while SSH authentication activity is recorded in the system authentication logs. Both log sources are ingested into Splunk under a single index to enable centralized analysis.

I first verified that Suricata alerts were successfully ingested into Splunk. The Suricata `eve.json` file was monitored and indexed using the `suricata:json` sourcetype.

After ingestion, I confirmed that JSON-formatted Suricata events were searchable in Splunk, indicating that network traffic data was being collected correctly.

The screenshot displays the Splunk Enterprise interface. At the top, the navigation bar includes 'splunk>enterprise', 'Apps', and various user and system icons. Below this, the 'Search & Reporting' section is active, showing a 'New Search' page. The search bar contains the query 'index=main sourcetype="suricata:json" | head 20'. The results show 20 events, with the first three displayed in a table format. The table has columns for 'Time' and 'Event'. The first event is a 'fileinfo' event from 11/29/25 3:23:30.603 AM. The second event is a 'flow' event from 11/29/25 3:23:29.909 AM. The third event is a 'http' event from 11/29/25 3:23:28.004 AM. The interface also shows a list of fields on the left, including 'SELECTED FIELDS' and 'INTERESTING FIELDS'.

i	Time	Event
>	11/29/25 3:23:30.603 AM	{ [-] app_proto: http dest_ip: 192.168.56.1 dest_port: 49574 event_type: fileinfo fileinfo: { [+] } flow_id: 1355498990707144 http: { [+] } in_iface: enp0s3 pkt_src: wire/pcap proto: TCP src_ip: 192.168.56.101 src_port: 8000 timestamp: 2025-11-29T03:23:30.603972+0000 }
>	11/29/25 3:23:29.909 AM	{ [-] app_proto: failed dest_ip: 224.0.0.252 dest_port: 5355 event_type: flow flow: { [+] } flow_id: 971577610885419 in_iface: enp0s3 proto: UDP src_ip: 192.168.56.1 src_port: 64851 timestamp: 2025-11-29T03:23:29.909149+0000 }
>	11/29/25 3:23:28.004 AM	{ [-] dest_ip: 192.168.56.101 dest_port: 8000 event_type: http flow_id: 1355498990707144 http: { [+] } }

Next, I verified ingestion of Linux SSH authentication logs from `/var/log/auth.log`. These logs were indexed using the `linux_secure` sourcetype. SSH login attempts, sudo activity, and other authentication-related events were visible in Splunk, confirming successful host-based log ingestion.

The screenshot displays the Splunk Enterprise web interface. At the top, the navigation bar includes 'splunk enterprise', 'Apps', and various user and system icons. Below this, a secondary navigation bar contains 'Search', 'Analytics', 'Datasets', 'Reports', 'Alerts', and 'Dashboards'. The 'Search' tab is active, leading to the 'New Search' page.

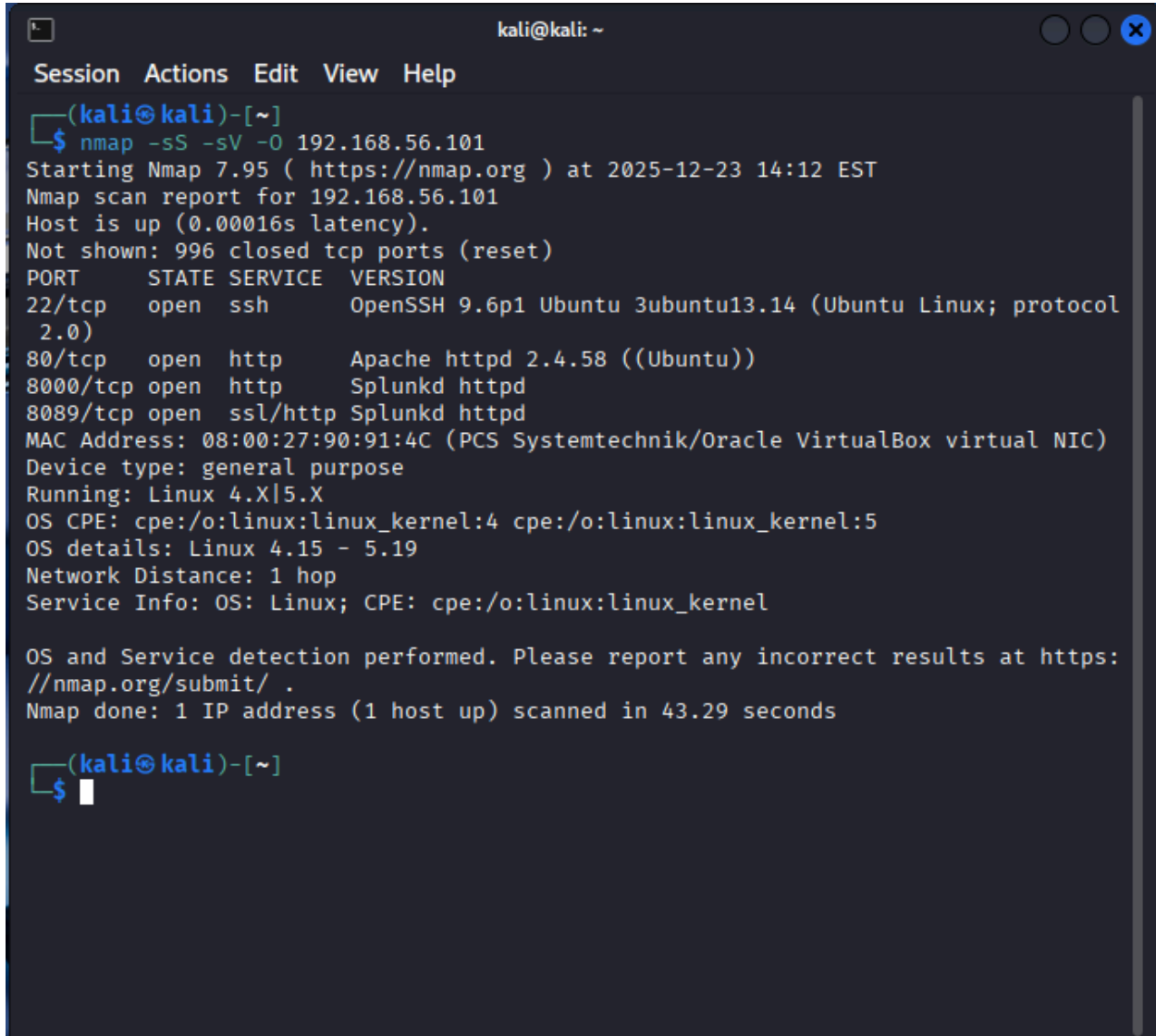
The search query is `index=main sourcetype="linux_secure"` with a 'head 20' limit. The time range is set to 'All time'. The search results show 20 events. The interface includes a timeline visualization at the top of the results pane, showing a series of green bars representing event occurrences over time. Below the timeline, there are controls for 'Format', 'Show: 20 Per Page', and 'View: List'.

The results are displayed in a table with columns for 'Time' and 'Event'. The events are sorted by time, showing a sequence of session openings and closings for user 'root' on host 'labserver'. The events include details such as the source path (`/var/log/auth.log`), sourcetype (`linux_secure`), and specific log messages from the `auth.log` file.

On the left side of the interface, there is a sidebar for field selection. It includes 'SELECTED FIELDS' (host, source, sourcetype) and 'INTERESTING FIELDS' (date_hour, date_mday, date_minute, date_month, date_second, date_wday, date_year, date_zone, index, linecount, pid, process, punct, splunk_server, timeendpos, timestartpos, uid). There are also options to 'Hide Fields', 'All Fields', and 'Extract New Fields'.

i	Time	Event
>	11/29/25 3:17:01.763 AM	2025-11-29T03:17:01.763273+00:00 labserver CRON[7072]: pam_unix(cron:session): session closed for user root host = labserver : source = /var/log/auth.log : sourcetype = linux_secure
>	11/29/25 3:17:01.751 AM	2025-11-29T03:17:01.751311+00:00 labserver CRON[7072]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0) host = labserver : source = /var/log/auth.log : sourcetype = linux_secure
>	11/29/25 3:15:01.738 AM	2025-11-29T03:15:01.738440+00:00 labserver CRON[5749]: pam_unix(cron:session): session closed for user root host = labserver : source = /var/log/auth.log : sourcetype = linux_secure
>	11/29/25 3:15:01.732 AM	2025-11-29T03:15:01.732703+00:00 labserver CRON[5749]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0) host = labserver : source = /var/log/auth.log : sourcetype = linux_secure
>	11/29/25 3:10:01.722 AM	2025-11-29T03:10:01.722000+00:00 labserver CRON[4295]: pam_unix(cron:session): session closed for user root host = labserver : source = /var/log/auth.log : sourcetype = linux_secure
>	11/29/25 3:10:01.717 AM	2025-11-29T03:10:01.717987+00:00 labserver CRON[4295]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0) host = labserver : source = /var/log/auth.log : sourcetype = linux_secure
>	11/29/25 3:09:21.233 AM	2025-11-29T03:09:21.233153+00:00 labserver sudo: pam_unix(sudo:session): session closed for user root host = labserver : source = /var/log/auth.log : sourcetype = linux_secure
>	11/29/25 3:09:05.607 AM	2025-11-29T03:09:05.607559+00:00 labserver mongod: looking for plugins in '/home/build/build-home/opt/mongo/openssl3/lib/sasl2', failed to open directory, error: No such file or directory host = labserver : source = /var/log/auth.log : sourcetype = linux_secure
>	11/29/25 3:09:05.604 AM	2025-11-29T03:09:05.604614+00:00 labserver mongod: looking for plugins in '/home/build/build-home/opt/mongo/openssl3/lib/sasl2', failed to open directory, error: No such file or directory host = labserver : source = /var/log/auth.log : sourcetype = linux_secure
>	11/29/25 3:09:01.709 AM	2025-11-29T03:09:01.709230+00:00 labserver CRON[3284]: pam_unix(cron:session): session closed for user root host = labserver : source = /var/log/auth.log : sourcetype = linux_secure
>	11/29/25 3:09:01.703 AM	2025-11-29T03:09:01.703838+00:00 labserver CRON[3284]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0) host = labserver : source = /var/log/auth.log : sourcetype = linux_secure
>	11/29/25 3:08:13.961 AM	2025-11-29T03:08:13.961464+00:00 labserver sudo: pam_unix(sudo:session): session opened for user root(uid=0) by lab (uid=1000) host = labserver : source = /var/log/auth.log : sourcetype = linux_secure
>	11/29/25 3:08:13.960 AM	2025-11-29T03:08:13.960105+00:00 labserver sudo: lab : TTY=ttty1 ; PWD=/home/lab ; USER=root ; COMMAND=/opt/splunk/bin/splunk restart --accept-license --answer-yes

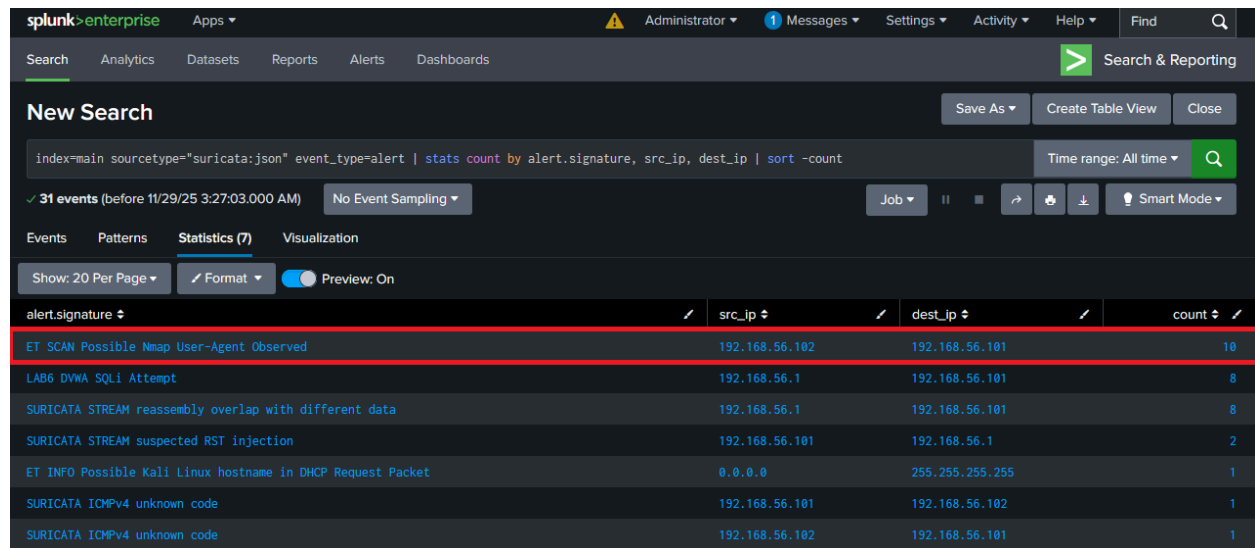
To simulate reconnaissance activity, I launched an Nmap scan from the Kali Linux attacker against the Ubuntu server. The scan included SYN scanning and service detection to enumerate open ports and running services. This attack represents the initial reconnaissance phase commonly observed in real-world intrusion attempts.

A screenshot of a terminal window titled 'kali@kali: ~'. The window has a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. The terminal shows the execution of an Nmap scan command: `nmap -sS -sV -O 192.168.56.101`. The output includes the Nmap version (7.95), the target IP (192.168.56.101), and a list of open ports with their corresponding services and versions. The scan was completed in 43.29 seconds.

```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ nmap -sS -sV -O 192.168.56.101  
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-23 14:12 EST  
Nmap scan report for 192.168.56.101  
Host is up (0.00016s latency).  
Not shown: 996 closed tcp ports (reset)  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.14 (Ubuntu Linux; protocol 2.0)  
80/tcp    open  http     Apache httpd 2.4.58 ((Ubuntu))  
8000/tcp  open  http     Splunkd httpd  
8089/tcp  open  ssl/http Splunkd httpd  
MAC Address: 08:00:27:90:91:4C (PCS Systemtechnik/Oracle VirtualBox virtual NIC)  
Device type: general purpose  
Running: Linux 4.X|5.X  
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5  
OS details: Linux 4.15 - 5.19  
Network Distance: 1 hop  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel  
  
OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 43.29 seconds  
  
(kali@kali)-[~]  
$
```

After executing the Nmap scan, I analyzed Suricata alerts in Splunk to identify scan-related detections. Suricata generated alerts such as *ET SCAN Possible Nmap User-Agent Observed*,

clearly indicating network reconnaissance activity originating from the attacker IP. The source and destination IP addresses confirmed the attack path.



The screenshot shows the Splunk Enterprise interface with a search query: `index=main sourcetype="suricata:json" event_type=alert | stats count by alert.signature, src_ip, dest_ip | sort -count`. The results are displayed in a table with columns: alert.signature, src_ip, dest_ip, and count. The first row is highlighted in red.

alert.signature	src_ip	dest_ip	count
ET SCAN Possible Nmap User-Agent Observed	192.168.56.102	192.168.56.101	10
LAB6 DYMA SQLi Attempt	192.168.56.1	192.168.56.101	8
SURICATA STREAM reassembly overlap with different data	192.168.56.1	192.168.56.101	8
SURICATA STREAM suspected RST injection	192.168.56.101	192.168.56.1	2
ET INFO Possible Kali Linux hostname in DHCP Request Packet	0.0.0.0	255.255.255.255	1
SURICATA ICMPv4 unknown code	192.168.56.101	192.168.56.102	1
SURICATA ICMPv4 unknown code	192.168.56.102	192.168.56.101	1

To safely test SSH brute-force behavior, I created a dedicated test account on the Ubuntu server. This ensured that the attack did not impact legitimate user accounts.

```
lab@labserver:~$ sudo adduser hydrauser
[sudo] password for lab:
info: Adding user `hydrauser' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `hydrauser' (1002) ...
info: Adding new user `hydrauser' (1002) with group `hydrauser (1002)' ...
info: Creating home directory `/home/hydrauser' ...
info: Copying files from `/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for hydrauser
Enter the new value, or press ENTER for the default
    Full Name []: hydrauser
    Room Number []: 203
    Work Phone []: 1112223333
    Home Phone []: 1112223333
    Other []:
Is the information correct? [Y/n] y
info: Adding new user `hydrauser' to supplemental / extra groups `users' ...
info: Adding user `hydrauser' to group `users' ...
lab@labserver:~$
```

Using the Kali attacker, I executed a Hydra attack against the SSH service on the Ubuntu server. Hydra attempted multiple password guesses for the test account, generating repeated

authentication failures. This simulates credential brute-force behavior commonly associated with unauthorized access attempts.

```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ hydra -l hydrauser -P smalllist.txt ssh://192.168.56.101 -t 4 -V  
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in mil  
itary or secret service organizations, or for illegal purposes (this is non-bindi  
ng, these *** ignore laws and ethics anyway).  
  
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-12-23 14:19:2  
5  
[DATA] max 4 tasks per 1 server, overall 4 tasks, 13 login tries (l:1/p:13), ~4 t  
ries per task  
[DATA] attacking ssh://192.168.56.101:22/  
[ATTEMPT] target 192.168.56.101 - login "hydrauser" - pass "123456" - 1 of 13 [ch  
ild 0] (0/0)  
[ATTEMPT] target 192.168.56.101 - login "hydrauser" - pass "password" - 2 of 13 [ch  
ild 1] (0/0)  
[ATTEMPT] target 192.168.56.101 - login "hydrauser" - pass "qwerty" - 3 of 13 [ch  
ild 2] (0/0)  
[ATTEMPT] target 192.168.56.101 - login "hydrauser" - pass "letmein" - 4 of 13 [c  
hild 3] (0/0)  
[ATTEMPT] target 192.168.56.101 - login "hydrauser" - pass "root" - 5 of 13 [chil  
d 0] (0/0)  
[ATTEMPT] target 192.168.56.101 - login "hydrauser" - pass "admin" - 6 of 13 [chi  
ld 1] (0/0)  
[ATTEMPT] target 192.168.56.101 - login "hydrauser" - pass "toor" - 7 of 13 [chil  
d 2] (0/0)  
[ATTEMPT] target 192.168.56.101 - login "hydrauser" - pass "12345" - 8 of 13 [chi  
ld 3] (0/0)  
[ATTEMPT] target 192.168.56.101 - login "hydrauser" - pass "123456789" - 9 of 13  
[child 0] (0/0)  
[ATTEMPT] target 192.168.56.101 - login "hydrauser" - pass "welcom" - 10 of 13 [c  
hild 1] (0/0)  
[ATTEMPT] target 192.168.56.101 - login "hydrauser" - pass "abc123" - 11 of 13 [c  
hild 3] (0/0)  
[ATTEMPT] target 192.168.56.101 - login "hydrauser" - pass "test123" - 12 of 13 [c  
hild 2] (0/0)  
[ATTEMPT] target 192.168.56.101 - login "hydrauser" - pass "dragon" - 13 of 13 [c  
hild 0] (0/0)  
1 of 1 target completed, 0 valid password found  
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-12-23 14:19:4
```

I analyzed SSH authentication logs in Splunk to detect brute-force activity. Failed password attempts were identified using the `linux_secure` sourcetype. Since SSH logs do not natively provide a structured source IP field, I used regular expression extraction to parse the attacker IP

and username from raw log messages. The results showed a high number of failed login attempts originating from the Kali attacker IP, confirming brute-force behavior.

The screenshot displays the Splunk Enterprise interface. At the top, the navigation bar includes 'splunk>enterprise', 'Apps', and various user and system settings. Below this, the 'Search & Reporting' section is active, showing a 'New Search' page. The search query is: `index=main sourcetype="linux_secure" "Failed password for" | rex "Failed password for (invalid user )?(?<user>\S+) from (?<src_ip>\d+\.\d+\.\d+\.\d+)" | stats count by src_ip, user | sort -count`. The results show 108 events. The 'Statistics (3)' tab is selected, displaying a table with columns: src_ip, user, and count. The table lists three entries, all from the IP 192.168.56.102, with counts of 56, 26, and 26 respectively. The second row, for user 'hydrauser', is highlighted with a red border.

src_ip	user	count
192.168.56.102	lab	56
192.168.56.102	hydrauser	26
192.168.56.102	root	26

splunk>enterpriseApps

AdministratorMessagesSettingsActivityHelpFind

SearchAnalyticsDatasetsReportsAlertsDashboards

Search & Reporting

New Search

Save AsCreate Table ViewClose

index=main sourcetype="linux_secure" "Failed password for" | rex "Failed password for (invalid user)?(?<user>\S+) from (?<src_ip>\d+\.\d+\.\d+\.\d+)" | search src_ip="192.168.56.102"Time range: All time

108 events (before 11/29/25 3:42:02.000 AM)No Event SamplingJobPauseRefreshDownloadSmart Mode

Events (108)PatternsStatisticsVisualization

Timeline formatZoom OutZoom to SelectionDeselect1 hour per column

FormatShow: 20 Per PageView: ListPrev123456Next

< Hide FieldsAll Fields

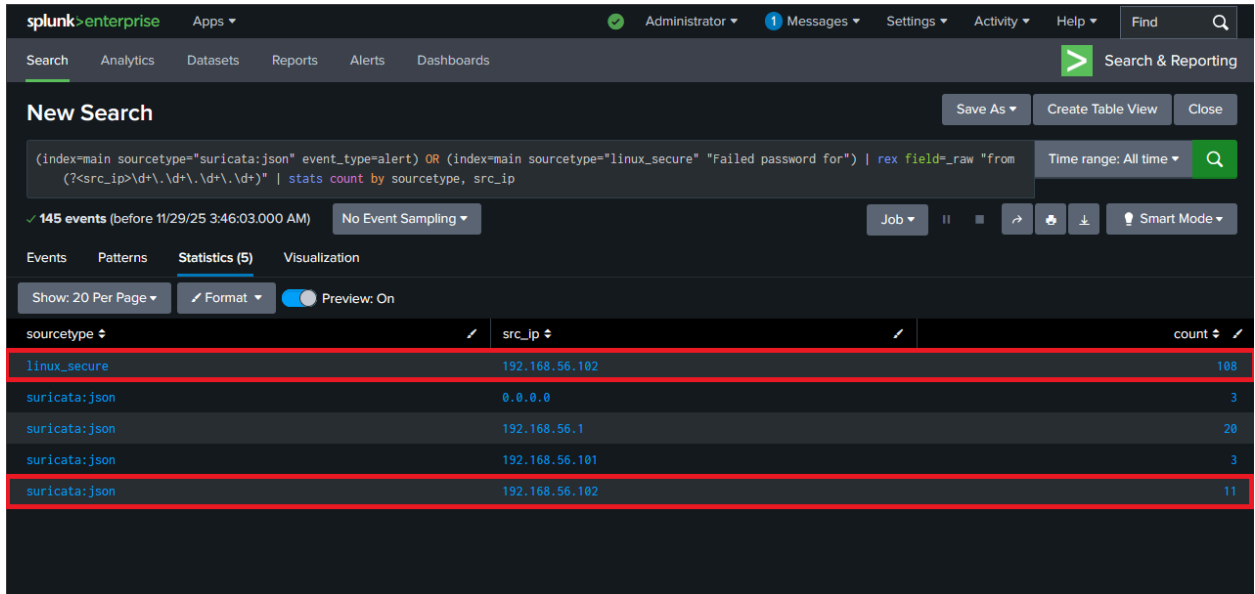
SELECTED FIELDS
a host 1
a source 1
a sourcetype 1

INTERESTING FIELDS
date_hour 4
date_mday 3
date_minute 9
a date_month 1
date_second 21
a date_wday 3
date_year 1
date_zone 1
a index 1
linecount 1
pid 75
a process 1
a punct 2
a splunk_server 1
a src_ip 1
timeendpos 1
timestartpos 1
a user 3

+ Extract New Fields

i	Time	Event
>	11/29/25 3:41:10.385 AM	2025-11-29T03:41:10.385888+00:00 labserver sshd[10271]: Failed password for hydrauser from 192.168.56.102 port 34868 host = labserver : source = /var/log/auth.log : sourcetype = linux_secure
>	11/29/25 3:41:08.307 AM	2025-11-29T03:41:08.307902+00:00 labserver sshd[10273]: Failed password for hydrauser from 192.168.56.102 port 34888 host = labserver : source = /var/log/auth.log : sourcetype = linux_secure
>	11/29/25 3:41:08.306 AM	2025-11-29T03:41:08.306072+00:00 labserver sshd[10274]: Failed password for hydrauser from 192.168.56.102 port 34894 host = labserver : source = /var/log/auth.log : sourcetype = linux_secure
>	11/29/25 3:41:08.305 AM	2025-11-29T03:41:08.305015+00:00 labserver sshd[10272]: Failed password for hydrauser from 192.168.56.102 port 34870 host = labserver : source = /var/log/auth.log : sourcetype = linux_secure
>	11/29/25 3:41:08.277 AM	2025-11-29T03:41:08.277901+00:00 labserver sshd[10271]: Failed password for hydrauser from 192.168.56.102 port 34868 host = labserver : source = /var/log/auth.log : sourcetype = linux_secure
>	11/29/25 3:41:05.648 AM	2025-11-29T03:41:05.648349+00:00 labserver sshd[10274]: Failed password for hydrauser from 192.168.56.102 port 34894 host = labserver : source = /var/log/auth.log : sourcetype = linux_secure
>	11/29/25 3:41:05.648 AM	2025-11-29T03:41:05.648065+00:00 labserver sshd[10272]: Failed password for hydrauser from 192.168.56.102 port 34870 host = labserver : source = /var/log/auth.log : sourcetype = linux_secure
>	11/29/25 3:41:05.647 AM	2025-11-29T03:41:05.647090+00:00 labserver sshd[10273]: Failed password for hydrauser from 192.168.56.102 port 34888 host = labserver : source = /var/log/auth.log : sourcetype = linux_secure
>	11/29/25 3:41:05.623 AM	2025-11-29T03:41:05.623122+00:00 labserver sshd[10271]: Failed password for hydrauser from 192.168.56.102 port 34868 host = labserver : source = /var/log/auth.log : sourcetype = linux_secure
>	11/29/25 3:41:01.609 AM	2025-11-29T03:41:01.609982+00:00 labserver sshd[10274]: Failed password for hydrauser from 192.168.56.102 port 34894 host = labserver : source = /var/log/auth.log : sourcetype = linux_secure
>	11/29/25 3:41:01.609 AM	2025-11-29T03:41:01.609631+00:00 labserver sshd[10273]: Failed password for hydrauser from 192.168.56.102 port 34888 host = labserver : source = /var/log/auth.log : sourcetype = linux_secure

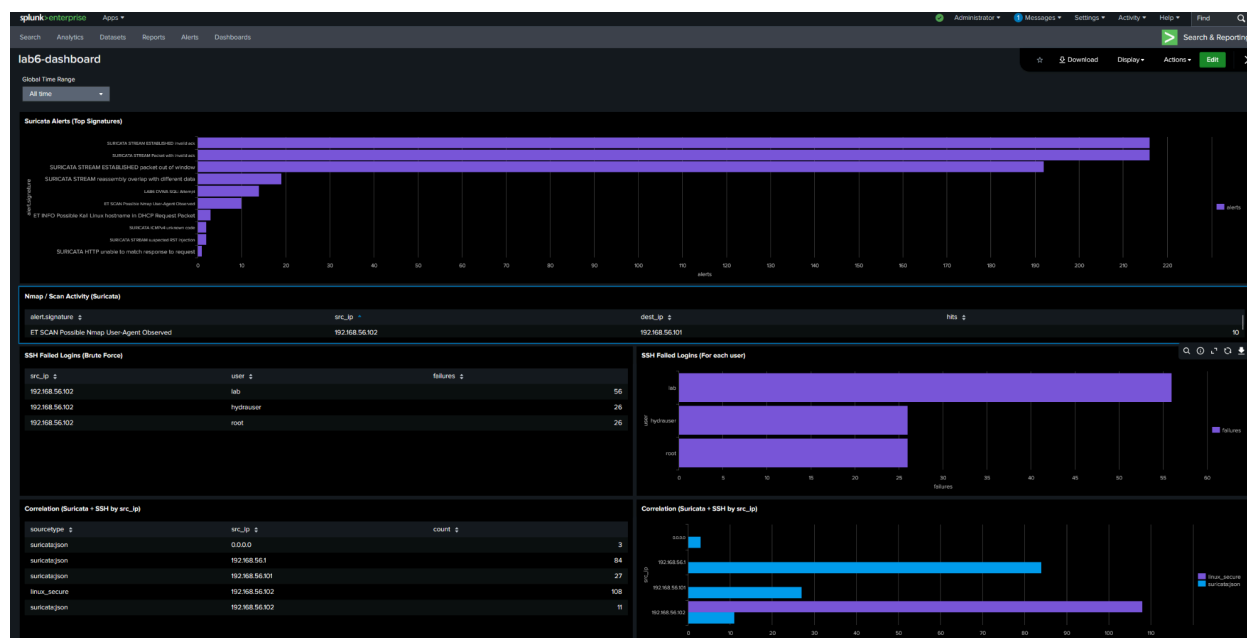
To mirror the correlation logic used in Lab 5, I correlated Suricata alerts and SSH authentication failures by attacker IP. By normalizing the source IP field across both network and host-based logs, I confirmed that the same attacker IP was responsible for both reconnaissance activity and SSH brute-force attempts. This correlation reconstructs a multi-stage attack sequence originating from a single source.



The screenshot displays the Splunk Enterprise interface. At the top, the navigation bar includes 'splunk>enterprise', 'Apps', and user roles like 'Administrator'. Below this, a 'New Search' section shows a search query: `(index=main sourcetype="suricata:json" event_type=alert) OR (index=main sourcetype="linux_secure" "Failed password for") | rex field=_raw "from (?<src_ip>\d+\.\d+\.\d+\.\d+)" | stats count by sourcetype, src_ip`. The results show 145 events. A table below the search bar lists the data:

sourcetype	src_ip	count
linux_secure	192.168.56.102	108
suricata:json	0.0.0.0	3
suricata:json	192.168.56.1	20
suricata:json	192.168.56.101	3
suricata:json	192.168.56.102	11

After validating individual detections, I created a Splunk dashboard to visualize the full detection workflow. The dashboard includes panels summarizing Suricata alerts, Nmap scan activity, SSH brute-force attempts, and correlation by attacker IP. Its structure closely mirrors the one used in Lab 5, with Splunk searches replacing Wazuh rules.



The results show a clear attack progression from reconnaissance to attempted access. Network scans were detected by Suricata, while repeated SSH authentication failures were recorded at the host level. Correlating these logs provides a coherent view of attacker behavior across multiple data sources.

This lab relies on manual SPL searches rather than automated alerts. Additionally, the environment represents a small-scale lab and does not reflect enterprise-level traffic volume or complexity. Also, it demonstrates that SIEM detection workflows are portable across platforms when designed correctly. By reproducing the same attack scenarios and detection logic used in Lab 5, I confirmed that Splunk can effectively ingest, detect, correlate, and visualize security events using both network and host-based logs.